# Supersede: when RAG cites the obsolete version

Measuring staleness in RAG, and a librarian agent that recovers document lineage

Carl Hatoum

Living draft. Every number is regenerated from `results/` (revision `0243817+dirty`, 2026-11-10).

**Abstract**

Retrieval-augmented generation (RAG) serves outdated documents: recent benchmarks show it across Wikipedia edits, medicine, law and software. Their fixes, such as time-decay ranking or recency re-ranking, rely on dates being present and right, and fail when they are not. We study the case they leave open: *the collection does not say which document replaces which.* On the IETF RFC series, whose index records 1,543 replacement links among 9,852 documents, we first confirm that lexical, dense and hybrid retrievers rank the obsolete version first for a large share of queries and that the error reaches answers when metadata is missing. We then infer the lineage itself, with every explicit cue removed, using a cascade: a cheap feature-based scorer for clear cases and a tool-using *librarian agent* (LangGraph, tools over MCP, local open-weight models) for the ambiguous ones. The benchmark, code and every result are public, and this draft is rebuilt from them.

**Key findings**

1. **Staleness, confirmed on a real lineage graph.** For questions whose answer changed between versions, the hybrid retriever puts the obsolete document first 65% of the time; without dates or version numbers, 21.7% of answers are stale (Figs. 2, 3).
2. **Known lineage removes it.** Promoting the current version cuts top-1 staleness to zero, and showing both versions with the old one marked brings stale answers to 0.0%.
3. **Cheap signals are not enough.** With every explicit cue removed, a similarity-and-date scorer closes 51% of the gap to perfect lineage, but simple newest-first ranking does about as well, and both degrade together when years are missing or wrong (Figs. 4, 5).
4. **Reading the ambiguous pairs helps; tools do not.** On 100 pairs the scorer finds ambiguous, a local 9.7B model reaches 0.76 balanced accuracy in one call and 0.75 as a tool-using agent, against 0.72 for the scorer; with half the years corrupted the model barely moves (0.74) while the scorer drops to 0.68. Used in a cascade, it raises the share of questions with the current version first from 55% to 67% (66% of the gap to perfect lineage closed). None of these differences is significant at this sample size (Figs. 7, 8).

## 1 The problem

Standards are revised, procedures reissued, policies replaced, and the old version usually stays in the collection. Nothing in a RAG pipeline [1] tells the retriever which version is in force, and the old document is often the *better* match: it has the canonical title and states the topic plainly, while the newer one may describe its changes. Figure 1 shows a real case from our benchmark.

We ask three questions:

**Q1.** How often does retrieval surface the obsolete version first? (§3)

**Q2.** Does it reach the generated answer, and can an answer policy contain it? (§4)

**Q3.** When lineage is not recorded, can we infer it, cheaply and with an agent? (§5–7)

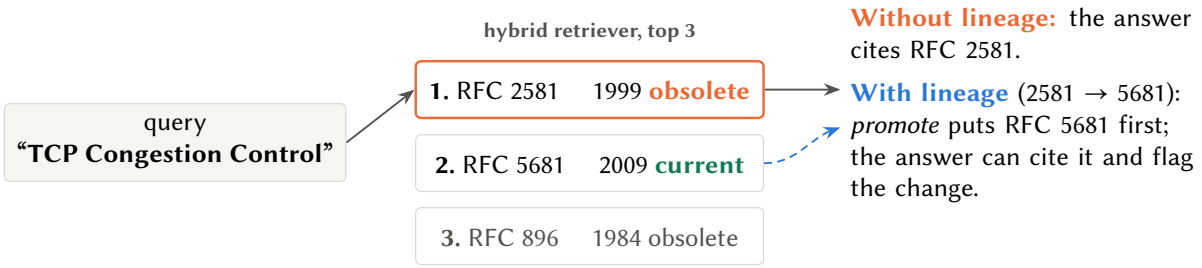



**Figure 1.** A real ranking from the benchmark. RFC 5681 replaced RFC 2581 in 2009, but the hybrid retriever ranks the obsolete document first. The same happens for TLS 1.2: the current version, RFC 9846, only comes fourth, behind two documents it replaced.

**How to read the numbers**

| | |
|---|---|
| **stale@1** | share of queries whose *top* result is an obsolete version (lower is better). |
| **current@1** | share of queries whose top result is a version in force (higher is better). |
| **lineage** | the graph of which document replaces which. *Oracle* lineage is the true one, from the index. |
| **gap closed** | how much of the improvement from no lineage to oracle lineage an inferred lineage recovers. |
| **balanced accuracy** | the mean of the hit rate on true replacements and on non-replacements; 0.5 is chance. |
| **[a, b]** | a 95% bootstrap confidence interval. Overlapping intervals mean the difference is not established. |

## 2 The benchmark

final · RFC snapshot e1669ddd5a87

**Documents and lineage.** We parse the RFC index (`rfc-index.xml`, built 2026-10-30): 9,852 RFCs, each with a title, year, keywords and, for 9,145 of them, an abstract. The index records *obsoletes* and *obsoleted-by*; taking a link from either side gives 1,543 replacement links over 1,384 obsolete documents, forming 800 lineages (the largest has 35 documents). The *current versions* of a document are the ends of its chain of replacements; a split can produce several. *Updates* relations (2,162) amend a document without replacing it and are not used as ground truth.

**Splits.** All queries and pairs of one lineage share a split (30% dev, 70% test), so no lineage is seen on both sides.

**Title queries.** For each link, the old title and the new title are each used as a query: 2,246 queries (1,511 test). Any current version is a correct result; any obsolete member of the lineage is stale.

**Change-aware questions.** Title queries are artificial, so we also generate natural questions whose answer *changed* between versions. For 300 sampled links, an LLM reads both abstracts and writes a question, the old and new answers, and verbatim evidence from each. Automatic checks reject evidence that is not verbatim, questions that name an RFC or give away the answer, and answers argued from silence. 69 questions pass (23.0%); in 224 cases the model found no substantive change and declined (Table 7 in the appendix).

The benchmark is public on Hugging Face (§10).

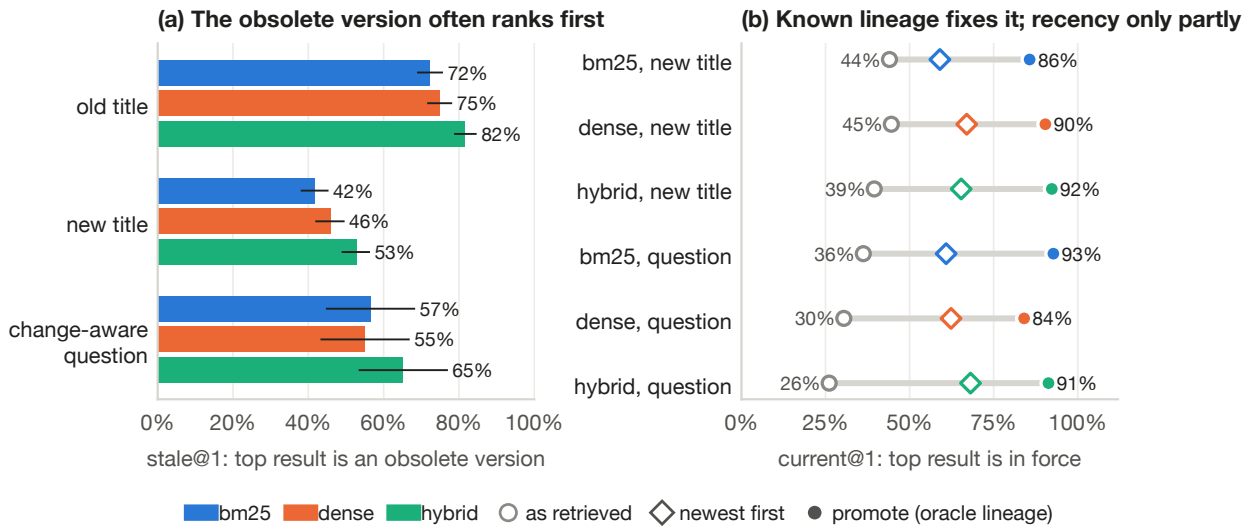



**Figure 2.** **(a)** Share of queries whose top result is obsolete, by query type and retriever, with 95% intervals. **(b)** Share whose top result is current, as retrieved (hollow) and after `promote` with oracle lineage (filled). Test split; questions use all splits ($n$=69). Full numbers: Tables 1–2.

## 3 Q1: does retrieval surface the obsolete version?

final

**Setup.** Three retrievers: BM25 [7], dense embeddings (`bge-small-en-v1.5` [8]), and their reciprocal rank fusion [9] ("hybrid"). Top 10. With oracle lineage we also test two fixes: `filter` drops obsolete documents; `promote` replaces each obsolete hit with its current version, at the same rank.

**Results.** Every retriever often ranks the obsolete version first (Fig. 2a): on new-title queries, 42% of the time with BM25, 46% with dense retrieval and 53% with the hybrid; on change-aware questions, 65% [54%, 77%] with the hybrid. The best topical matcher is the most stale, because the old document *is* a good match for the topic, as HoH found for dense retrievers on Wikipedia [12]. With oracle lineage, `promote` raises the hybrid's current@1 from 39% to 92% on new titles (Fig. 2b).

**Is newest-first enough?.** A fix without lineage is to prefer recent documents. We re-rank the hybrid's top 50 by year within a window (`newest_first`) or by score $\times e^{-\lambda \cdot \text{age}}$ (`time_decay`, as in HoH [12]), with the window and $\lambda$ tuned on dev. With correct years, this halves staleness but stops well short of lineage: on new-title queries the hybrid's current@1 rises from 39% to 65% (newest-first) and 60% (time decay), against 92% with `promote`; on questions, 68% against 91%. A newer document is not necessarily the replacement: it may be an unrelated, companion or partial update. Recency also needs correct years, which Section 6 relaxes.

> **Takeaway.** Staleness is a property of the collection, not of a weak retriever: better retrieval makes it worse. Knowing the lineage removes it almost completely, which is why the rest of the paper is about recovering the lineage.

## 4 Q2: does it reach the answer?

final · human check of the judge pending

**Setup.** For each question, the hybrid retriever's results go to an answering model (`claude-sonnet-5-5`, top 5) under three policies, all using oracle lineage except the first: `naive` (context as retrieved); `latest_only` (current versions only); `show_conflict` (`promote` order, obsolete documents kept and marked

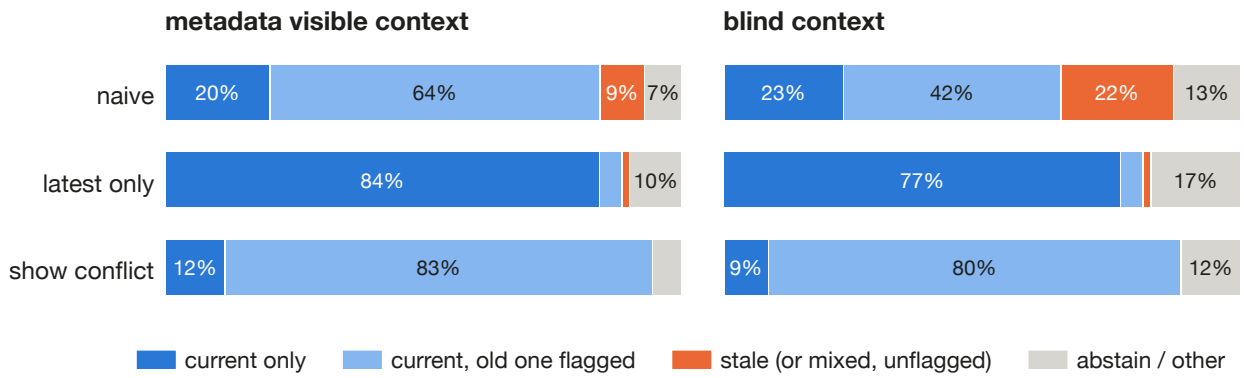



**Figure 3.** What the answers say, per policy ($n$=69 questions each). "Current, old one flagged" gives the new answer and mentions the old one as outdated. Table 3 in the appendix gives intervals.

as replaced). A judge model (`claude-haiku-5-5`) compares each answer with the reference old and new answers. A *stale* outcome gives the old answer without flagging it as outdated.

In the standard context each passage shows its year and RFC number, which a strong model can use to pick the newer one by itself. The *blind* variant hides years and numbers, as in collections without reliable metadata.

**Results.** With metadata visible, `naive` gives a stale answer 8.7% of the time; the model mostly resolves versions itself. Blind, this rises to 21.7% [13.0%, 31.9%] (Fig. 3). `show_conflict` keeps it at 0.0% and gives the most current answers (88.4% blind), while telling the reader what changed.

**What the model knows without documents.** Asked the same questions with no documents, the answering model gives the current answer 80% of the time [70%, 88%] and the old one 14%: it has read most of these RFCs. So retrieval does not supply knowledge the model lacks; it *overrides* knowledge the model has. Of the 55 questions it gets right closed-book, blind `naive` RAG turns 11 (20%) into stale answers (11% with metadata visible); `latest_only` and `show_conflict` turn none. Where it does not know (14 questions), current documents rescue it, 79% of the time with `show_conflict` blind. This is a lower bound for less familiar collections, such as internal procedures, which no model has read.

**Judge reliability.** A second model labelled 100 answers without seeing the judge's labels: Cohen's $\kappa$ = 0.85 [11] (91 of 100 identical). This is agreement between two models; a human check is pending ($\kappa$ = ??, $n$ = ??).

> **Takeaway.** Where metadata is missing, about one answer in five is stale, often where the model knew better without retrieval. Showing both versions, with the old one marked, fixes it, but it needs the lineage.

## 5 Q3a: inferring lineage cheaply

final · librarian v0

**Hiding the answer.** Many abstracts say outright "This document obsoletes RFC 2581", and the index's lineage fields are hidden from every method. Three settings: `declared` keeps the text; `masked` replaces every RFC number with a neutral token; `strict` also removes sentences that announce a replacement. `strict` matches collections that never declare it.

**Method.** Candidate pairs are each document's 10 nearest neighbours by abstract and by title embedding, older first (120,390 pairs, covering 89% of the true links). Each pair gets cheap features (title and abstract similarity, year gap, version cues such as "bis" or "version 2", shared keywords, neighbour rank) and a score from a logistic regression trained on the dev split, with a threshold chosen on dev. Each old document

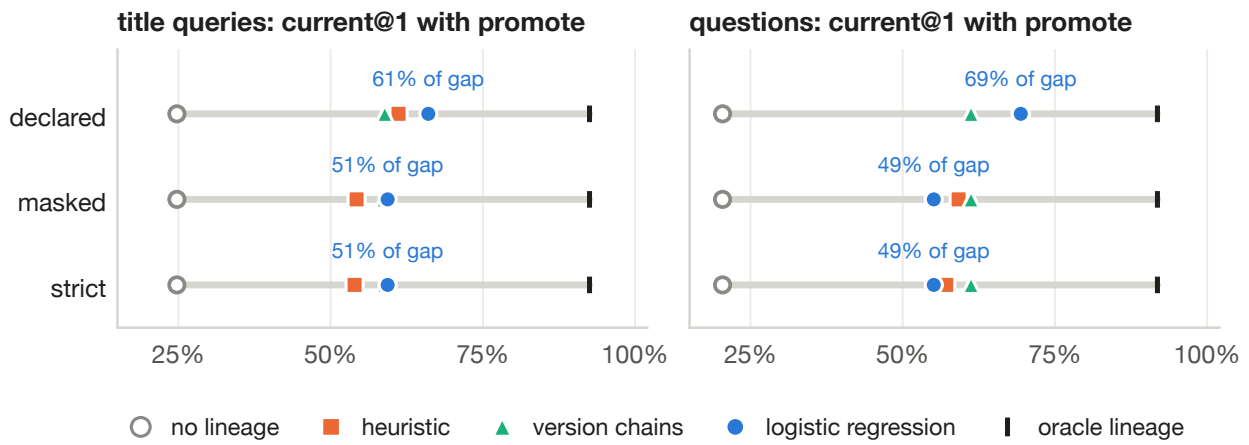



**Figure 4.** current@1 after `promote` using inferred lineage, between no lineage (hollow) and oracle lineage (bar); squares: heuristic, triangles: version chains, dots: logistic regression. Labels: share of the gap closed by the logistic regression. Edge precision, recall and F1 in Table 4.

keeps its best replacement above the threshold. As a learning-free baseline in the style of Modani et al. [17], *version chains* link candidate pairs whose title or abstract similarity exceeds a threshold (tuned on dev), treat each connected group as a version family, and chain its members in publication order.

**Results.** The logistic regression's edge F1 drops from 0.663 (declared) to 0.539 (strict), while version chains hold at 0.624: once explicit cues are gone, the learned scorer is no better than clustering similar documents. Downstream the two are close: 51% and 50% of the gap closed on title queries, 49% and 57% on questions (Fig. 4). Its mistakes cluster in a band of high but uncertain scores, mostly pairs with near-identical titles that are companions rather than successors.

> **Takeaway.** Cheap signals recover half of what lineage is worth, about as much as preferring recent documents (Section 6). The other half sits in ambiguous pairs that need reading, which is the agent's job.

## 6 When years are missing or wrong

final

**Setup.** Recency baselines need correct years, and so does our cheap scorer, which uses years to orient a pair (older→newer) and as a feature. We remove or shift (by 1–10 years) the year of a random share of documents, re-tune the recency parameters, and re-fit the scorer on the corrupted data, over 3 seeds per level. Document numbers are never used for ordering, so they cannot leak the true order. Oracle lineage needs no years and is the ceiling.

**Results.** With correct years, newest-first reaches 57% current@1 on title queries and the inferred lineage 58%. With half of the years missing or wrong, newest-first falls to 42% and time decay to 41%, while the inferred lineage keeps 44% (Fig. 5); on questions the corresponding values are 48%, 49% and 41%.

> **Takeaway.** The cheap scorer is *not* more robust than recency: it orients pairs by year, so it degrades at the same rate, and with correct years simple recency already matches it (title queries) or beats it (questions). Both stay far below oracle lineage at every noise level. Closing that gap needs judgements made from the content, which is what the agent is for; §7 tests whether reading holds up when years are wrong.

## 7 Q3b: a librarian agent for the uncertain cases

100 pairs · one corruption seed

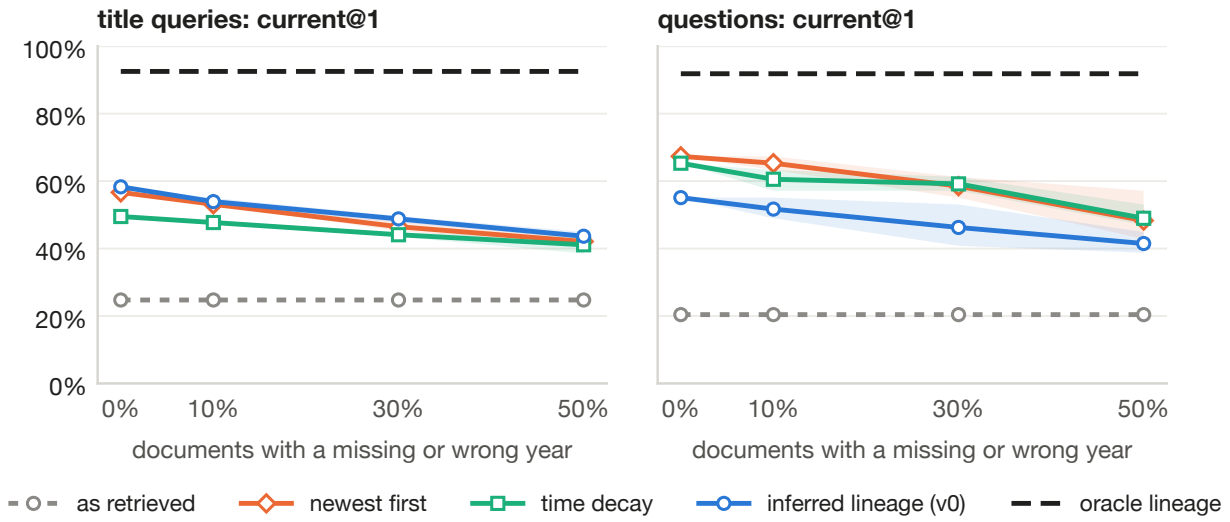



**Figure 5.** current@1 of the hybrid's top 10 on test queries as a growing share of years is removed or shifted. Lines are means over seeds; bands span the seeds. Inferred lineage uses the `strict` setting.

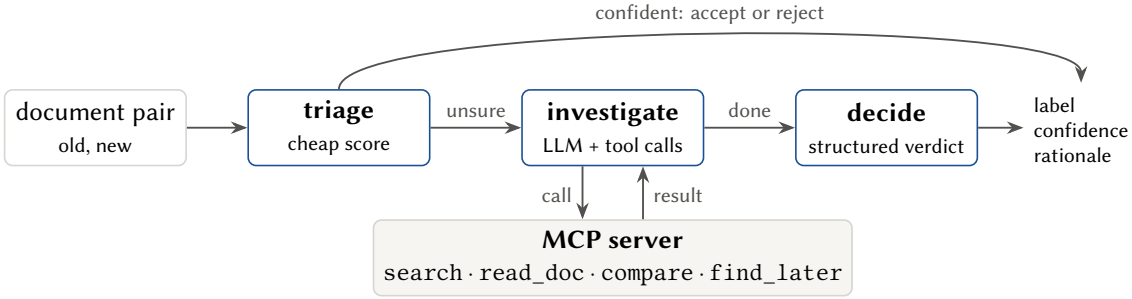



**Figure 6.** The librarian agent. Only pairs in the cheap scorer's uncertain band reach the language model, so the agent's cost scales with ambiguity, not corpus size.

**Design.** The agent decides whether a newer document replaces an older one (Fig. 6). It is a LangGraph state machine: *triage* settles pairs the cheap scorer is sure about; *investigate* lets the model call tools for up to 8 steps, ReAct-style [20]; *decide* returns a structured verdict (*supersedes*, *amends*, *conflicts*, *unrelated*) with a confidence and a one-line rationale. The tools come from an MCP server, the same one used in deployment, and never reveal lineage: `search`, `read_doc`, `compare` and `find_later` (documents on the same topic published later: the candidate replacements). Models run locally through Ollama at temperature 0.

**Evaluation.** We take test pairs whose cheap-scorer probability falls in the uncertain band [0.8, 0.98) (`strict` setting, 2,735 pairs) and draw a seeded sample of 100: 50 true replacements and 50 non-replacements. The sample is nested (its first 20 and 50 pairs are the samples used for repeated runs). Each method is scored *on the same pairs*, against two ground truths: *strict* (the index records the link directly) and *lineage* (a later version through a chain also counts: if A→B→C, then "C replaces A" is right for a reader). The sample is balanced but the band is not (14% of band pairs are true links), so we report the precision expected in use, estimated from the hit rates (Table 5). Paired differences are tested with an exact McNemar test and a paired bootstrap of the balanced-accuracy difference.

**What we compare.** Three ways of using the same model (`qwen3.5`, 9.7B, temperature 0) and the same prompt (v2), to tell what the tools add: *LLM only* reads both documents (the same text `read_doc` returns) and decides in one call; *agent without `find_later`* has `search`, `read_doc` and `compare`; *agent, all tools* adds `find_later`. The cheap scorer uses its own tuned threshold (0.94). Prompt v2 was written after looking at 20 earlier pairs, ten of which are in this sample (Appendix B).

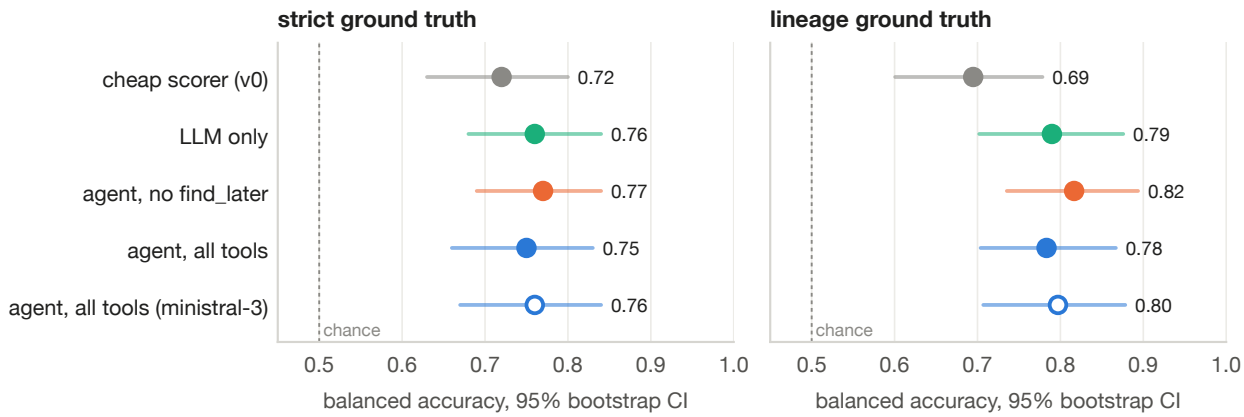



**Figure 7.** Balanced accuracy on the same 100 band pairs, with 95% stratified bootstrap intervals. Grey: the cheap scorer; colours: the same Qwen 3.5 model with no tools (aqua), three tools (orange) and all four tools (blue); hollow: Ministral 3 with all tools.

**Results.** All three model-based methods do somewhat better than the cheap scorer, and none of the differences is significant (Fig. 7). Under the strict ground truth, the full agent reaches 0.75 [0.66, 0.83] against 0.72 [0.63, 0.80] (difference +0.03 [-0.08, +0.15], McNemar $p = 0.75$). Under the lineage ground truth the gap widens: the agent without `find_later` reaches 0.82 against 0.69 (McNemar $p = 0.02$). Many of the model's "false" replacements are later issues of the same series, such as two editions of *Internet Official Protocol Standards* several versions apart: wrong for the index, right for a reader.

The tools do not help. A single call without tools (0.76) is as accurate as the full agent (difference -0.01, $p = 1.00$) at a quarter of the cost: 35 s per pair against 151 s, and 595 input tokens against 5,437 (Fig. 8, Table 6). The two make the same call on most pairs. The agent does follow the procedure (it calls `compare` and `find_later` on every pair), but what it finds rarely changes its mind.

A second model, Ministral 3 (8B) with the same prompt and all tools, lands in the same place: 0.76 [0.67, 0.84] (McNemar against the scorer $p = 0.64$), at 66 s per pair. It calls `compare` on every pair and never `find_later`, so it decides from the two documents alone, like the no-tool variant.

**Run-to-run stability.** Sampling at temperature 0.3, 3 runs on the first 50 pairs give balanced accuracies of 0.74–0.78 for the full agent and 0.72–0.74 without tools; the label is the same in every run for 80% and 90% of pairs. Sampling noise is small next to the confidence intervals, so the conclusions above do not depend on one run.

All model-based methods say "replaces" too readily: the full agent does so on 59% of a balanced sample, with true-positive rate 0.84 and true-negative rate 0.66. In the real band, where only 14% of pairs are true links, this gives an expected precision of 0.29, below the scorer's 0.36. Whether the extra recall outweighs the extra false links downstream is measured next.

**When years are wrong.** We rerun the same 100 pairs with half of all years removed or shifted (the corruption of §6; seed 0), shown to the model through its tools and used by the cheap scorer, retrained on the corrupted years. The scorer drops from 0.72 to 0.68 [0.60, 0.76]: it misses more true links (true-positive rate 0.62 to 0.44), because it orients and scores pairs by year. The model barely moves: 0.76 to 0.74 without tools, 0.75 to 0.76 with them. The gap to the scorer widens (+0.08 [-0.03, +0.18] for the agent, McNemar $p = 0.27$) but is not significant at 100 pairs. This is the direction the cascade needs: the step that reads is the one that does not lean on dates.

**Effect on retrieval.** The cascade keeps the scorer's decision outside the band and uses the model's verdict inside it. Only band pairs whose older document appears in the hybrid top 10 of a test question can change a ranking; there are 203, and the no-tool model decided all of them (coverage 100%). With `promote`, the current version comes first for 67% of the 49 test questions, against 55% with the scorer alone and 92% with perfect lineage: the cascade closes 66% of the gap, the scorer 49%. The difference

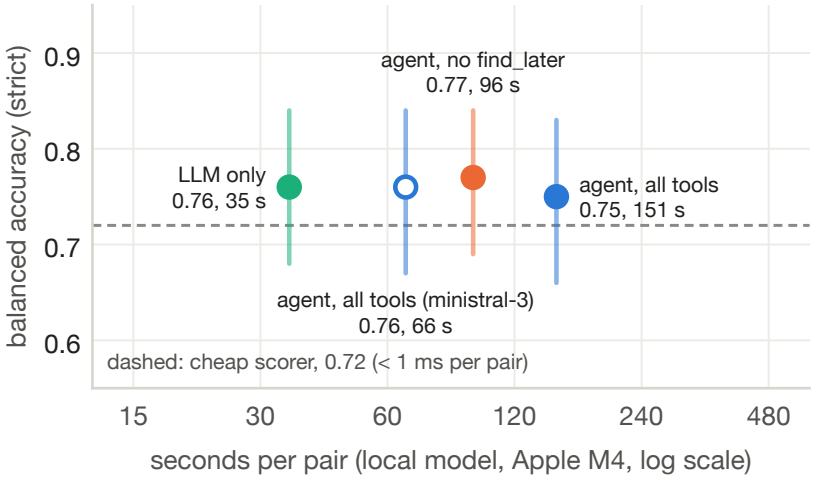



**Figure 8.** Accuracy against cost per pair on a laptop (Apple M4). Bars: 95% bootstrap intervals; hollow: Ministral 3. The dashed line is the cheap scorer.

(+12 points, bootstrap 95% CI [+2, +22]) comes from 7 questions fixed against 1 broken (exact McNemar $p = 0.07$). The extra false links cost less than the extra true ones gain. With the full agent deciding the same pairs instead, the cascade closes 60% of the gap (current version first for 63%; 5 questions fixed against 1 broken, $p = 0.22$): no better than one call, at about four times the cost.

> **Takeaway.** On pairs where cheap signals are ambiguous, reading the two documents helps: a little on its own, more in retrieval through the cascade, and it holds up when years are wrong, where the scorer and recency degrade. The investigation tools add cost but no accuracy, so a single model call is the better design here. Every gain is in the expected direction and none is yet significant at this sample size.

## 8 Related work

**Outdated documents in RAG.** HoH [12] builds 96k questions from monthly Wikipedia diffs, keeps current and outdated passages together, and shows that outdated passages hurt more than distracting ones and that stronger retrievers fetch more of them; mitigations rely on modification times. Stale-Document Poisoning [13] measures how often outdated evidence overturns an answer a model gets right closed-book (17–91% for open models, 317 reversals in four domains); its recency re-ranker helps only when dates are correct, and with wrong dates it stops selecting the current document at all. MemStrata [14] retires superseded facts in agent memory using ingestion order as time, and warns that textual markers such as "deprecated" let baselines read the answer, which our masked and strict settings rule out. Churn-Bench [15] measures freshness errors of agents over enterprise data with timestamped change logs. All of these assume time metadata or change logs; *we target collections where lineage is unrecorded and dates are unreliable*, and infer the lineage.

**Version detection.** Modani et al. [17] detect versions and order them in a Wikipedia-derived collection. GVD [16] groups enterprise documents into version families and classifies rule-level relations (duplicate, contradiction, refinement) without an LLM, on 120 confidential documents, without a downstream RAG evaluation. We infer directed replacement links, including chains, on a public benchmark, and measure their effect on retrieval and answers; our second stage is an agent with tools.

**Knowledge conflicts and time.** Conflict benchmarks such as ConflictBank [18] and WikiContradict [19] study disagreements between memory and context or between passages; time-sensitive QA [2, 3, 4] studies questions whose answer changes; and knowledge-conflict work asks how models arbitrate [5, 6]. Here the conflict has a ground truth, the lineage. LLM judges [10] need validation, which we report as agreement statistics; the agent follows the ReAct pattern of interleaved reasoning and tool calls [20].

## 9 Limitations and next steps

- **One domain.** RFCs, with short abstracts. Aviation advisory circulars are next. The answering model knows the current answer to 80% of the questions without documents (§4); collections no model has read will test retrieval harder.
- **Few questions, model-checked.** 69 change-aware questions, validated against a second model rather than people (a human check is pending; HoH and Stale-Document Poisoning report human checks). A larger, head-only generation is planned.
- **One answering model** so far; open-weight answering models are in progress.
- **Agent evidence is limited.** One prompt, 100 pairs and 49 test questions: intervals are about ±0.08 wide, so differences under 0.1 are not detectable. One corruption seed for the year-noise runs.
- **Deployment.** The serving code (HTTP search with inferred lineage, the cascade, MCP and A2A interfaces) is in the repository; the cloud deployment is in progress and will be reported with latency and cost.

## 10 Reproducibility and release

Code, results and this paper: `https://github.com/CarlHatoum/supersede`. `make paper` regenerates every number, table and figure from `results/`. LLM calls for questions, answers and judging are cached, so `make reproduce` rebuilds them offline; agent decisions are logged per pair with tool traces. The benchmark is at `https://huggingface.co/datasets/hatoum/supersede-rfc`. RFC abstracts ship verbatim, as the IETF Trust Legal Provisions allow unmodified, attributed excerpts but not modified text; the `masked` and `strict` views are derived locally by a script shipped with the data.

# A Full tables

| retriever | query | policy | stale@1↓ | current@1↑ | current@10↑ | MRR↑ |
|---|---|---|---|---|---|---|
| bm25 | new title | none | 0.417 | 0.440 | 0.931 | 0.628 |
| | | filter | 0.000 | 0.750 | 0.931 | 0.821 |
| | | promote | 0.000 | 0.857 | 0.983 | 0.905 |
| | | newest_first | 0.161 | 0.590 | 0.931 | 0.703 |
| | | time_decay | 0.124 | 0.579 | 0.906 | 0.704 |
| | old title | none | 0.722 | 0.152 | 0.790 | 0.394 |
| | | filter | 0.000 | 0.568 | 0.790 | 0.651 |
| | | promote | 0.000 | 0.874 | 0.986 | 0.916 |
| | | newest_first | 0.274 | 0.448 | 0.790 | 0.542 |
| | | time_decay | 0.352 | 0.357 | 0.778 | 0.508 |
| dense | new title | none | 0.459 | 0.446 | 0.928 | 0.643 |
| | | filter | 0.000 | 0.803 | 0.928 | 0.855 |
| | | promote | 0.000 | 0.904 | 0.992 | 0.938 |
| | | newest_first | 0.186 | 0.670 | 0.928 | 0.756 |
| | | time_decay | 0.140 | 0.609 | 0.913 | 0.720 |
| | old title | none | 0.749 | 0.170 | 0.793 | 0.415 |
| | | filter | 0.000 | 0.620 | 0.793 | 0.686 |
| | | promote | 0.000 | 0.916 | 0.996 | 0.947 |
| | | newest_first | 0.321 | 0.496 | 0.793 | 0.578 |
| | | time_decay | 0.321 | 0.384 | 0.774 | 0.517 |
| hybrid | new title | none | 0.528 | 0.395 | 0.937 | 0.621 |
| | | filter | 0.000 | 0.809 | 0.937 | 0.861 |
| | | promote | 0.000 | 0.923 | 0.994 | 0.950 |
| | | newest_first | 0.183 | 0.653 | 0.937 | 0.750 |
| | | time_decay | 0.169 | 0.597 | 0.941 | 0.737 |
| | old title | none | 0.816 | 0.111 | 0.810 | 0.389 |
| | | filter | 0.000 | 0.615 | 0.810 | 0.689 |
| | | promote | 0.000 | 0.927 | 0.996 | 0.953 |
| | | newest_first | 0.315 | 0.486 | 0.810 | 0.576 |
| | | time_decay | 0.327 | 0.401 | 0.820 | 0.556 |

**Table 1.** Retrieval on test-split title queries ($n$=727 new-title, 784 old-title). `filter` and `promote` use oracle lineage.

| retriever | policy | stale@1↓ [95% CI] | current@1↑ |
|---|---|---|---|
| bm25 | none | 0.565 [0.449, 0.681] | 0.362 [0.261, 0.478] |
| | filter | 0.000 [0.000, 0.000] | 0.768 [0.667, 0.870] |
| | promote | 0.000 [0.000, 0.000] | 0.928 [0.870, 0.986] |
| | newest_first | 0.217 [0.116, 0.319] | 0.609 [0.493, 0.725] |
| | time_decay | 0.174 [0.087, 0.275] | 0.638 [0.522, 0.754] |
| dense | none | 0.551 [0.435, 0.667] | 0.304 [0.203, 0.420] |
| | filter | 0.000 [0.000, 0.000] | 0.681 [0.565, 0.783] |
| | promote | 0.000 [0.000, 0.000] | 0.841 [0.754, 0.913] |
| | newest_first | 0.188 [0.101, 0.275] | 0.623 [0.507, 0.739] |
| | time_decay | 0.101 [0.043, 0.174] | 0.609 [0.493, 0.725] |
| hybrid | none | 0.652 [0.536, 0.768] | 0.261 [0.159, 0.362] |
| | filter | 0.000 [0.000, 0.000] | 0.783 [0.681, 0.870] |
| | promote | 0.000 [0.000, 0.000] | 0.913 [0.841, 0.971] |
| | newest_first | 0.188 [0.101, 0.275] | 0.681 [0.565, 0.783] |
| | time_decay | 0.145 [0.072, 0.232] | 0.667 [0.551, 0.783] |

**Table 2.** Retrieval on change-aware questions ($n$=69, all splits).

| context | policy | stale answer↓ [95% CI] | current↑ | conflict flagged | abstain |
|---|---|---|---|---|---|
| standard | `naive` | 8.7% [2.9%, 15.9%] | 84.1% | 63.8% | 5.8% |
| | `latest_only` | 1.4% [0.0%, 4.3%] | 88.4% | 4.3% | 7.2% |
| | `show_conflict` | 0.0% [0.0%, 0.0%] | 94.2% | 82.6% | 4.3% |
| blind | `naive` | 21.7% [13.0%, 31.9%] | 65.2% | 42.0% | 13.0% |
| | `latest_only` | 1.4% [0.0%, 4.3%] | 81.2% | 4.3% | 14.5% |
| | `show_conflict` | 0.0% [0.0%, 0.0%] | 88.4% | 79.7% | 10.1% |

**Table 3.** Answer outcomes ($n$=69), hybrid retrieval, top-5 context. "Current" includes answers that flag the old version.

| setting | method | edge P | edge R | edge F1 | title queries: current@1 (gap) | questions: current@1 (gap) |
|---|---|---|---|---|---|---|
| declared | no lineage | – | – | – | 0.248 (0%) | 0.204 (0%) |
| | heuristic | 0.325 | 0.541 | 0.406 | 0.612 (54%) | 0.694 (69%) |
| | version chains | 0.792 | 0.514 | 0.623 | 0.589 (50%) | 0.612 (57%) |
| | logreg | 0.775 | 0.578 | 0.663 | 0.660 (61%) | 0.694 (69%) |
| | oracle | – | – | – | 0.925 (100%) | 0.918 (100%) |
| masked | no lineage | – | – | – | 0.248 (0%) | 0.204 (0%) |
| | heuristic | 0.288 | 0.449 | 0.351 | 0.543 (44%) | 0.592 (54%) |
| | version chains | 0.783 | 0.515 | 0.621 | 0.588 (50%) | 0.612 (57%) |
| | logreg | 0.593 | 0.481 | 0.531 | 0.594 (51%) | 0.551 (49%) |
| | oracle | – | – | – | 0.925 (100%) | 0.918 (100%) |
| strict | no lineage | – | – | – | 0.248 (0%) | 0.204 (0%) |
| | heuristic | 0.286 | 0.444 | 0.348 | 0.539 (43%) | 0.571 (51%) |
| | version chains | 0.783 | 0.519 | 0.624 | 0.589 (50%) | 0.612 (57%) |
| | logreg | 0.596 | 0.492 | 0.539 | 0.594 (51%) | 0.551 (49%) |
| | oracle | – | – | – | 0.925 (100%) | 0.918 (100%) |

**Table 4.** Lineage inference: edge scores on test pairs, and downstream current@1 with `promote` (share of gap closed in brackets).

| truth | method | TPR | TNR | bal. acc. [95% CI] | est. prec. | s/pair |
|---|---|---|---|---|---|---|
| strict | cheap scorer (v0) | 0.62 | 0.82 | 0.72 [0.63, 0.80] | 0.36 | – |
| | agent, all tools, `qwen3.5-hermes` | 0.84 | 0.66 | 0.75 [0.66, 0.83] | 0.29 | 151 |
| | LLM only (no tools), `qwen3.5-hermes` | 0.88 | 0.64 | 0.76 [0.68, 0.84] | 0.29 | 35 |
| | agent without `find_later`, `qwen3.5-hermes` | 0.92 | 0.62 | 0.77 [0.69, 0.84] | 0.29 | 96 |
| | agent, all tools, `ministral-3` | 0.82 | 0.70 | 0.76 [0.67, 0.84] | 0.31 | 66 |
| lineage | cheap scorer (v0) | 0.57 | 0.82 | 0.69 [0.60, 0.78] | 0.51 | – |
| | agent, all tools, `qwen3.5-hermes` | 0.84 | 0.73 | 0.78 [0.70, 0.87] | 0.50 | 151 |
| | LLM only (no tools), `qwen3.5-hermes` | 0.88 | 0.70 | 0.79 [0.70, 0.88] | 0.49 | 35 |
| | agent without `find_later`, `qwen3.5-hermes` | 0.93 | 0.70 | 0.82 [0.74, 0.89] | 0.51 | 96 |
| | agent, all tools, `ministral-3` | 0.82 | 0.77 | 0.80 [0.71, 0.88] | 0.54 | 66 |

**Table 5.** Agent variants and cheap scorer on the same 100 band pairs. Estimated precision uses the band's real share of true links (0.14 strict, 0.25 lineage).

| method | s/pair | model calls | tool calls | input tok. | output tok. | says replaces |
|---|---|---|---|---|---|---|
| agent, all tools, `qwen3.5-hermes` | 151 | 4.0 | 2.0 | 5,437 | 803 | 59% |
| LLM only (no tools), `qwen3.5-hermes` | 35 | 1.0 | 0.0 | 595 | 186 | 62% |
| agent without `find_later`, `qwen3.5-hermes` | 96 | 3.2 | 1.3 | 3,259 | 481 | 65% |
| agent, all tools, `ministral-3` | 66 | 3.0 | 1.0 | 2,568 | 382 | 56% |

**Table 6.** Cost per pair (means; local Qwen 3.5 on an Apple M4) and the share of pairs labelled "replaces".

| outcome | pairs |
|---|---|
| skipped_by_model | 224 |
| accepted | 69 |
| evidence_current_not_verbatim | 2 |
| question_leaks_reference | 2 |
| answer_from_silence | 1 |
| evidence_stale_not_verbatim | 1 |
| question_reveals_answer | 1 |

**Table 7.** Outcome of question generation per sampled link.

## B Agent prompt development and traces

Prompts were developed on an earlier, non-nested 20-pair sample. Prompt v1 never searched: it read both documents, compared them and accepted look-alike pairs. Prompt v2 added `find_later`, a fixed procedure (compare, then look for later versions) and a confidence rubric. Ministral 3 (8B) with prompt v2 called `compare` once per pair and never `find_later`. These runs are kept for the record; the results in §7 use the 100-pair sample.

| truth | method | prompt | TPR | TNR | bal. acc. [95% CI] | est. prec. |
|---|---|---|---|---|---|---|
| strict | cheap scorer (v0) | – | 0.40 | 0.90 | 0.65 [0.50, 0.85] | 0.40 |
| | agent: `ministral-3` | v2 | 0.90 | 0.60 | 0.75 [0.55, 0.90] | 0.27 |
| | agent: `qwen3.5-hermes` | v1 | 1.00 | 0.70 | 0.85 [0.70, 0.95] | 0.35 |
| | agent: `qwen3.5-hermes` | v2 | 0.90 | 0.90 | 0.90 [0.75, 1.00] | 0.60 |
| lineage | cheap scorer (v0) | – | 0.31 | 0.86 | 0.58 [0.40, 0.74] | 0.41 |
| | agent: `ministral-3` | v2 | 0.85 | 0.71 | 0.78 [0.56, 0.96] | 0.49 |
| | agent: `qwen3.5-hermes` | v1 | 0.92 | 0.86 | 0.89 [0.71, 1.00] | 0.68 |
| | agent: `qwen3.5-hermes` | v2 | 0.77 | 1.00 | 0.88 [0.73, 1.00] | 1.00 |

| model | prompt | setting | n | failed | steps | s/pair | labels |
|---|---|---|---|---|---|---|---|
| `qwen3.5-hermes` | agent-v1 | strict | 20 | 0 | 3.0 | 122 | amends 2, supersedes 13, unrelated 5 |
| `qwen3.5-hermes` | agent-v2 | strict | 20 | 0 | 3.0 | 118 | amends 2, supersedes 10, unrelated 8 |
| `ministral-3` | agent-v2 | strict | 20 | 0 | 2.0 | 69 | amends 2, supersedes 13, unrelated 5 |

### `qwen3.5-hermes`, agent-v1

- **correct, supersedes**: `RFC332 -> RFC342` (gold supersedes; verdict `supersedes`, confidence 0.95).
  → `read_doc({"doc_id": "RFC332"})`
  → `read_doc({"doc_id": "RFC342"})`
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  *Both documents share the exact same title ('Network Host Status') and year (1972). In technical standards collections, when two documents with identical titles exist but are distinguished as 'older' and 'newer' (even if the publication year is the same or very close), the newer identifier typically represents a revision or replacement of the older one. The comparison tool confirmed they cover the same topic. Since RFC342 is designated as the 'NEWER' document in the pair, it supersedes RFC332.*

- **correct, not**: `RFC4145 -> RFC4572` (gold not; verdict `unrelated`, confidence 0.95).
  → `read_doc({"doc_id": "RFC4145"})`
  → `read_doc({"doc_id": "RFC4572"})`
  → `compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})`
  *RFC4145 (2005) defines media transport over plain TCP, while RFC4572 (2006) defines media transport over TLS. Although both use SDP and share similar attribute names ('setup', 'connection'), they address different underlying protocols (TCP vs. TLS). RFC4572 does not replace or supersede RFC4145; instead, they are complementary standards for different security requirements. Therefore, they are unrelated in the sense that neither supersedes the other.*

- **error**: `RFC266 -> RFC370` (gold not; verdict `supersedes`, confidence 0.95).
  → `read_doc({"doc_id": "RFC266"})`
  → `read_doc({"doc_id": "RFC370"})`
  → `compare({"doc_a": "RFC266", "doc_b": "RFC370"})`
  *The documents share identical titles ('Network host status' vs 'Network Host Status') and cover the same topic. The newer document (RFC370, 1972) is a direct update to the older one (RFC266, 1971). In technical standards, when a new version with the same title replaces an older one, it supersedes the previous version.*

### qwen3.5-hermes, agent-v2

- **correct, supersedes**: RFC3016 -> RFC6416 (gold supersedes; verdict supersedes, confidence 0.95).
  → compare({"doc_a": "RFC3016", "doc_b": "RFC6416"})
  → find_later({"doc_id": "RFC3016", "k": 5})
  *RFC6416 explicitly identifies itself as a 'necessary revision' of RFC3016 in its abstract. It states that it updates and completes the specification while discussing backward compatibility with the older document. The scope (RTP Payload Format for MPEG-4 Audio/Visual Streams) is identical, and the newer document corrects misalignments found in the older one, which is the definition of superseding in this context.*

- **correct, not**: RFC4145 -> RFC4572 (gold not; verdict unrelated, confidence 0.95).
  → compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})
  → find_later({"doc_id": "RFC4145", "k": 5})
  *RFC4145 defines media transport over plain TCP, while RFC4572 defines media transport over TLS (Transport Layer Security). Although both are SDP extensions for connection-oriented transport, they address different underlying protocols with different security properties. One does not replace the other; instead, they serve as companion specifications allowing implementers to choose between unencrypted and encrypted transport mechanisms based on their needs. The fact that RFC8122 (a later update to the TLS specification) exists further confirms that RFC4572 was an early version of the TLS SDP spec, not a successor to the plain TCP spec.*

- **error**: RFC332 -> RFC342 (gold supersedes; verdict unrelated, confidence 0.95).
  → compare({"doc_a": "RFC332", "doc_b": "RFC342"})
  → find_later({"doc_id": "RFC332", "k": 5})
  *Both documents share the exact same title ('Network Host Status') and were published in the same year (1972). The comparison tool indicates no abstract overlap (both lack abstracts) but 100% title word overlap. The 'find_later' query for RFC332 returned other RFCs from 1972, none of which were RFC342, suggesting they are not in a direct versioning chain where one replaces the other. Documents with identical titles from the same era often represent companion documents (e.g., different profiles or specific aspects) rather than a successor relationship. Since neither clearly restates and revises the scope of the other to act as a replacement, and they appear to be parallel works on the same topic, they are classified as unrelated in terms of supersession.*

### ministral-3, agent-v2

- **correct, supersedes**: RFC3016 -> RFC6416 (gold supersedes; verdict supersedes, confidence 0.95).
  → compare({"doc_a": "RFC3016", "doc_b": "RFC6416"})
  *RFC6416 explicitly states it is a 'necessary revision' of RFC3016, summarizes changes, and addresses backward compatibility issues. The scope remains identical (RTP payload formats for MPEG-4 Audio/Visual streams), and the newer document restates/revises the older one's content while clarifying limitations and correcting misalignments. This is a direct replacement, not merely an extension or companion document.*

- **correct, not**: RFC4145 -> RFC4572 (gold not; verdict amends, confidence 0.9).
  → compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})
  *RFC4572 explicitly states it 'extends and updates' a prior reference (likely RFC4145) by adding TLS security to TCP-based media transport in SDP. The newer document introduces a new protocol identifier ('TCP/TLS') and a fingerprint attribute, but it does not restate or revise the entire scope of RFC4145. Instead, it adds a new mechanism (TLS) to the same transport framework. The older document remains valid for plain TCP transport, and the newer one complements it rather than replacing it entirely. This is a clear case of amendment, not supersession.*

- **error**: `RFC332 -> RFC342` (gold supersedes; verdict `unrelated`, confidence 0.0).
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  *RFC332 and RFC342 share the same title and year (1972), meaning neither is newer than the other. Since supersession requires a later document to replace an earlier one, they cannot be in a superseding relationship. The lack of a chronological gap or explicit revision history makes this a definitive conclusion.*

## C Provenance

| section | source (under `results/` unless stated) |
|---|---|
| Retrieval staleness (title queries) | `rfc_staleness_test.json` |
| Retrieval staleness (change-aware questions) | `rfc_questions_all.json` |
| Question generation | `data/rfc/questions_manifest.json` |
| Answer staleness (standard) | `rfc_answer_staleness_hybrid.json` |
| Answer staleness (blind) | `rfc_answer_staleness_hybrid_blind.json` |
| Judge agreement (model rater) | `rfc_answer_staleness_hybrid_labels_claude-opus-5-5.csv` |
| Lineage inference (librarian v0) | `rfc_lineage_inference.json` |
| Recency vs. lineage under year noise | `rfc_recency_noise.json` |
| Closed-book baseline | `rfc_closed_book.json` |
| Agent: qwen3.5-hermes, agent-v1 | `agent/history/rfc_agent_strict_ollama-edtorre-qwen3-5-hermes_agent-v1.json` |
| Agent: qwen3.5-hermes, agent-v2 | `agent/history/rfc_agent_strict_ollama-edtorre-qwen3-5-hermes_agent-v2.json` |
| Agent: ministral-3, agent-v2 | `agent/history/rfc_agent_strict_ollama-ministral-3-8b_agent-v2.json` |
| Agent variants (100 pairs, strict) | `agent/compare_strict_n100.json` |
| Agent variants (100 pairs, strict-noise50) | `agent/compare_strict-noise50_n100.json` |
| Cascade downstream (questions, full) | `agent/downstream_questions_full.json` |
| Cascade downstream (questions, llm only) | `agent/downstream_questions_llm_only.json` |

## References


[1] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

[2] W. Chen, X. Wang, W. Y. Wang. A dataset for answering time-sensitive questions. *NeurIPS Datasets and Benchmarks*, 2021.

[3] J. Kasai et al. RealTime QA: What's the answer right now? *NeurIPS Datasets and Benchmarks*, 2023.

[4] B. Dhingra et al. Time-aware language models as temporal knowledge bases. *TACL*, 2022.

[5] S. Longpre et al. Entity-based knowledge conflicts in question answering. *EMNLP*, 2021.

[6] J. Xie et al. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. *ICLR*, 2024.

[7] S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in IR*, 2009.

[8] S. Xiao et al. C-Pack: Packaged resources to advance general Chinese embedding. arXiv:2309.07597, 2023.

[9] G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *SIGIR*, 2009.

[10] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.

[11] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 1960.

[12] J. Ouyang, T. Pan, M. Cheng, R. Yan, Y. Luo, J. Lin, Q. Liu. HoH: A dynamic benchmark for evaluating the impact of outdated information on retrieval-augmented generation. arXiv:2503.04800, 2025.

[13] M. S. Ahmed, L. Galke Poech, R. Röttger. Stale-document poisoning: When outdated retrieval overrides correct model answers. arXiv:2609.31342, 2026.

[14] Anonymous. Temporal validity in retrieval memory: Eliminating stale-fact errors for AI agents over evolving knowledge. arXiv:2606.26511, 2026.

[15] V. K. Singh, P. Priyam, G. Bhowmick. ChurnBench: A drift-aware benchmark demonstrating that refresh scheduling, not cache age, governs staleness in agentic AI. arXiv:2609.11515, 2026.

[16] M. Sediqin, S. Dalmia, S. Thoppanahalli, A. Mukherji. GVD: Governed versioning and deduplication for document repositories. arXiv:2609.17696, 2026.

[17] N. Modani, A. Maurya, G. Verma, I. Nair, V. Patil, A. Kanfade. Detecting document versions and their ordering in a collection. *WISE*, 2021.

[18] Z. Su et al. ConflictBank: A benchmark for evaluating the influence of knowledge conflicts in LLM. arXiv:2408.12076, 2024.

[19] Y. Hou et al. WikiContradict: A benchmark for evaluating LLMs on real-world knowledge conflicts from Wikipedia. *NeurIPS Datasets and Benchmarks*, 2024.

[20] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.